

LiveMCPBench: Can Agents Navigate an Ocean of MCP Tools?

Guozhao Mo*
University of Chinese Academy of
Sciences
Beijing, China
moguozhao2024@iscas.ac.cn

Wenliang Zhong
Institute of Software, Chinese
Academy of Sciences
Beijing, China
zhongwenliang2024@iscas.ac.cn

Jiawei Chen
Qianhao Yuan
Institute of Software, Chinese
Academy of Sciences
Beijing, China

Xuanang Chen
Institute of Software, Chinese
Academy of Sciences
Beijing, China
chenxuanang@iscas.ac.cn

Yaojie Lu
Institute of Software, Chinese
Academy of Sciences
Beijing, China
luyaojie@iscas.ac.cn

Hongyu Lin
Institute of Software, Chinese
Academy of Sciences
Beijing, China
hongyu@iscas.ac.cn

Ben He
University of Chinese Academy of
Sciences
Beijing, China
benhe@ucas.ac.cn

Xianpei Han
Institute of Software, Chinese
Academy of Sciences
Beijing, China
xianpei@iscas.ac.cn

Le Sun
Institute of Software, Chinese
Academy of Sciences
Beijing, China
sunle@iscas.ac.cn

Abstract

Model Context Protocol (MCP) has become a key infrastructure for connecting LLMs with external tools, scaling to 10,000+ MCP servers with diverse tools. Unfortunately, there is still a large gap between real-world MCP usage and current evaluation: they typically assume single-server settings and directly inject tools into the model's context, bypassing the challenges of large-scale retrieval and multi-tool composition. To bridge this gap, we propose **LiveMCPBench**, which evaluates 95 real-world daily tasks explicitly constructed to stress diverse tools and scaled multi-server routing. The benchmark includes a ready-to-deploy tool suite of 70 servers with 527 tools, ensuring reproducibility without scattered API configuration. We further introduce an LLM-as-a-Judge evaluation framework that directly verifies task outcomes, handling dynamic data sources and multiple valid solution paths. We benchmark 12 state-of-the-art LLMs and observe a substantial performance gap: while Claude-Sonnet-4 reaches 78.95% task success, most models achieve only 30–50%. Our analysis reveals that active tool composition strongly correlates with task success, whereas retrieval errors account for nearly half of all failures—highlighting retrieval as the dominant bottleneck. Together, these results provide the first large-scale, reproducible diagnosis of MCP agent capabilities and point towards future research on improving retrieval robustness

and encouraging effective tool composition. Our code and data are publicly available at <https://icip-cas.github.io/LiveMCPBench>.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; • **General and reference** → **Evaluation**.

Keywords

Model Context Protocol, Scalable MCP-use, Benchmark

ACM Reference Format:

Guozhao Mo, Wenliang Zhong, Jiawei Chen, Qianhao Yuan, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2018. LiveMCPBench: Can Agents Navigate an Ocean of MCP Tools?. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 18 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Tool-use agents powered by large language models (LLMs) are emerging as a critical step toward general intelligence [23, 30]. The Model Context Protocol [1, MCP] has rapidly become the core infrastructure for connecting LLMs with external tools [5]. Its ecosystem is rapidly scaling to more than 10,000 servers with diverse functionalities [12]. Meanwhile, pretrained models are being equipped with the ability to invoke MCP servers directly [24]. MCP has greatly expanded the boundaries of tool-use agents, enabling them to tackle increasingly complex real-world tasks [26]. As a result, the ability to use MCP effectively has become a key criterion of an agent's capability.

Despite the rapid expansion of MCP, existing benchmarks fall short of capturing its real-world challenges, as shown in Figure 1. Traditional tool-use benchmarks such as API-Bank [14] and Tool-Bench [22] rely on simulated or unstable APIs, resulting in less realistic and unreliable tasks. For example, 55.6% of APIs in Tool-Bench are no longer available [11]. In contrast, MCP provides a

*All authors are also affiliated with Chinese Information Processing Laboratory, Institute of Software, Chinese Academy of Sciences, Beijing, China

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

more stable invocation interface through a package-like management model, where servers can be reliably maintained and provide both local and online functionality. MCP adopts a server-tool architecture, where each server exposes multiple related tools with contextual descriptions, providing richer semantics. This structure requires agents to develop retrieval and composition abilities that go beyond conventional tool search. However, current MCP benchmarks remain narrow in scope. For example, MCPBench [18] covers only 10 servers and directly injects tools into the LLM’s context, bypassing large-scale retrieval and multi-tool composition entirely. Furthermore, most existing MCP benchmarks are still conducted in static environments, making them unable to evaluate agents’ performance in dynamic, large-scale MCP ecosystems.

These limitations lead to two central research questions:

- *RQ1: How can we systematically evaluate an agent’s retrieval and composition capabilities in large-scale, diverse MCP ecosystems?*
- *RQ2: How can we design evaluation methodologies that are both scalable and reproducible under dynamic, real-world settings?*

To address these questions, we introduce **LiveMCPBench**, a benchmark for realistic, large-scale MCP evaluation. LiveMCPBench covers six practical domains and consists of 95 multi-step, daily tasks that explicitly test retrieval and composition abilities. We focus on daily scenarios because most MCP servers are designed to provide up-to-date information (e.g., news), making them a natural and challenging frontier for assessing MCP-use capabilities. To facilitate reproducibility and ease of use, we provide **LiveMCPTool**, a curated collection of 70 MCP servers with 527 tools, packaged as a ready-to-use environment that removes the need for configuring numerous scattered API keys. For evaluation, we propose **LiveMCPEval**, an automatic evaluation framework based on the LLM-as-a-Judge [36], enabling scalable assessment even under dynamic data sources (e.g., evolving weather or news) and supporting multiple valid solution trajectories. Crucially, it supports scalable and open-ended evaluation, allowing seamless expansion as new MCP servers and tasks are introduced, without costly manual annotation.

Building on LiveMCPBench, we develop **MCP Copilot Agent**, a ReACT agent [34] capable of retrieving and composing tools across a large MCP toolset. We evaluate 12 leading LLMs on the benchmark and find a substantial performance gap: while Claude-Sonnet-4 achieves a 78.95% task success rate, the majority of widely used frontier models remain in the 30–50% range. By analyzing agent behaviors, we observe that active tool composition strongly correlates with task success, indicating that models capable of flexibly combining tools are more reliable. A detailed error analysis further shows that nearly half of all failures stem from tool retrieval errors, establishing retrieval as the primary bottleneck in large-scale MCP usage. These findings highlight the urgent need for future systems that jointly **improve tool composition** and **mitigate retrieval errors**. To further validate our evaluation pipeline, we conduct human annotation of agent trajectories and assess human–model agreement. DeepSeek-V3 attains an agreement rate of 81.05%, demonstrating the reliability of our evaluation protocol.

In summary, our key contributions are:

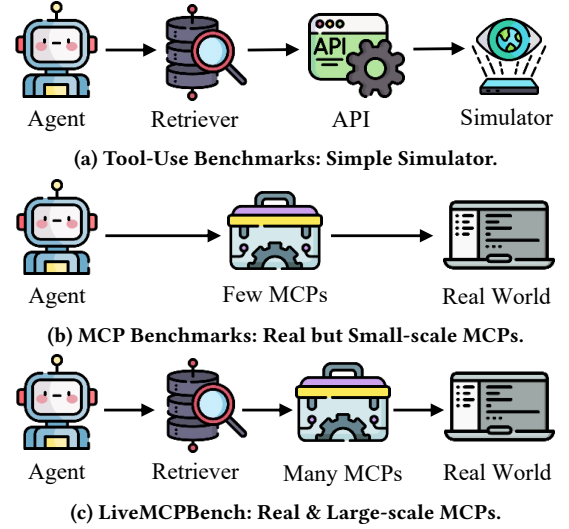


Figure 1: LiveMCPBench focuses on daily task resolution using a large-scale MCP toolset.

- We introduce LiveMCPBench, a benchmark with 95 multi-step daily tasks that test agents’ large-scale retrieval and multi-tool composition abilities in large-scale MCP ecosystems.
- We provide a reproducible toolset (70 MCP servers, 527 tools) together with an LLM-as-a-Judge automatic evaluation framework, enabling scalable, automated, and human-aligned assessment in dynamic environments.
- Our empirical study offers a large-scale diagnosis of MCP agent capabilities, highlighting tool retrieval and composition as unsolved challenges and laying the groundwork for future research.

2 LiveMCPBench

We present LiveMCPBench (Figure 2), a benchmark designed to evaluate the ability of agent systems to retrieve appropriate tools from a large-scale MCP toolset for accomplishing general-purpose daily tasks. Building such a benchmark requires addressing three key challenges:

- **Designing representative tasks** that faithfully capture and expose the unique characteristics and inherent complexities of large-scale MCP ecosystems.
- **Constructing a comprehensive, reproducible toolset** that remains both large and functionally complete while being easy to deploy for consistent and fair benchmarking.
- **Automating robust evaluation** of agent performance on dynamic, evolving online tasks, where task context and available tools may change over time.

In this section, we first detail our task construction methodology. Then, we describe the collection process of the LiveMCPTool. Next, we introduce LiveMCPEval, an LLM-as-a-Judge evaluation framework that enables robust and scalable assessment. Finally, we build MCP Copilot Agent, which serves as our baseline approach.

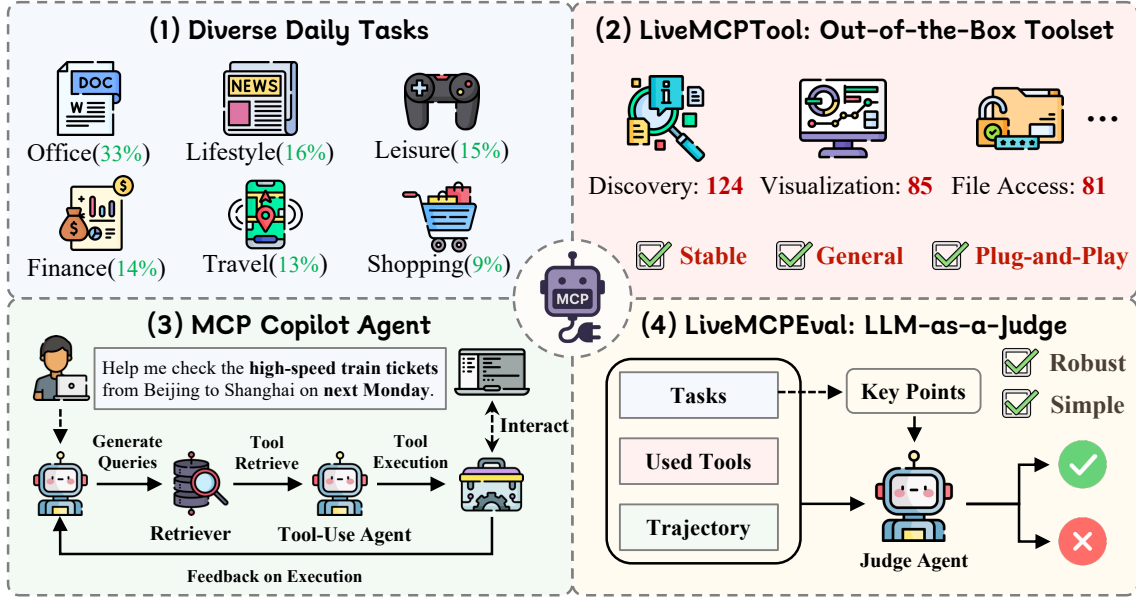


Figure 2: LiveMCPBench comprises four parts: (1) Diverse Daily Task, a collection of 95 daily tasks; (2) LiveMCPTool, a large-scale MCP toolset; (3) MCP Copilot Agent, a ReACT-based agent supporting retrieval and multi-turn tool invocation across the MCP toolset; (4) LiveMCPEval, an automated LLM-as-a-Judge evaluation system for accurately assessing dynamic tasks.

2.1 Task Construction

To ensure that LiveMCPBench reflects the practical strengths of MCP servers, we design tasks grounded in everyday scenarios where these servers are most impactful. Since many MCP servers are oriented toward real-time information retrieval (e.g., news, stock prices, weather updates) or local office productivity (e.g., spreadsheet analysis, scheduling), daily-life settings naturally highlight their distinctive capabilities. By anchoring tasks in such contexts, we capture both the immediacy and utility that characterize MCP-driven interactions.

The resulting benchmark spans six daily domains: *Office* (e.g., spreadsheet analysis), *Lifestyle* (e.g., news retrieval), *Leisure* (e.g., video game inquiries), *Finance* (e.g., stock price monitoring), *Travel* (e.g., ticket search), and *Shopping* (e.g., product recommendations). Tasks in these domains are inherently dynamic, often depending on evolving contexts such as monitoring fluctuating markets. They are also long-horizon, as solving them typically requires coordinating multiple tools rather than a single call. At the same time, the tasks retain genuine utility, reflecting realistic and meaningful user needs that go beyond artificial benchmarks.

The task creation process employed a rigorous two-stage methodology involving two groups of computer science students serving as task *proposers* and *validators*. Each group consisted of three members to ensure both diversity of perspectives and reliability of annotation. *Proposers* first generated scenario-specific tasks based on personal experience, with LLM-assisted ideation permitted but strictly vetted for authenticity. Each proposer then interacted with our toolset to complete their proposed task, meticulously annotating key points to preserve the task’s compositional depth. *Validators* subsequently scrutinized both the task design and corresponding

toolchain invocations, eliminating duplicates while enforcing quality standards. In total, *proposers* initially produced 300 candidate tasks across six domains. After iterative validation and refinement, this pipeline yielded 95 high-fidelity daily tasks (see Appendix E for annotation principles and distribution statistics).

2.2 LiveMCPTool Collection

While prior study [12] suggests the existence of over 10,000 MCP servers, creating a practical and accessible toolset remains nontrivial due to critical usability constraints. The predominant challenge stems from dependency fragmentation: the majority of MCP servers necessitate proprietary API keys or integrations with third-party services, rendering them impractical for a standardized toolset. To address this, we introduce a rigorously validated methodology for constructing a high-quality, dependency-free MCP toolset—prioritizing reproducibility and broad applicability. Our approach first aggregates 5,588 server configurations from a MCP Marketplace¹, then systematically filters out key-dependent servers to eliminate access barriers.

Beyond accessibility, we ensure the toolset’s representativeness through structured curation and expert annotation. Tools are taxonomically organized into five functional categories (Discovery, Visualization, File Access, Location, and Miscellaneous), followed by manual vetting to exclude low-quality implementations. To ensure reproducibility, we package all collected tools in a docker environment, providing a stable and consistent tool setup. This two-stage pipeline yields 70 MCP servers providing 527 tools, each verified for standalone functionality and categorical relevance. By decoupling the toolset from external dependencies, our collection

¹mcp.so

Table 1: Comparison with existing work. Plug & Play denotes the toolset requires no extra API keys. Dynamic denotes the task depending on evolving contexts. Syn. denotes tasks generated by LLMs. MCP-Zero is not a benchmark, but contains a toolset.

	Tools				Tasks			
	Stable	Servers	Tools	Plug & Play	Type	Number	Dynamic	Evaluation
<i>Tool-Use Benchmarks</i>								
API-Bank [14]	✓	8	73	✓	Real	314	✗	Rule
ToolBench [22]	✗	49	3,451	✓	Syn.	126,486	✓	LLM
τ -bench [33]	✓	2	28	✓	Real	165	✗	Rule
<i>MCP Benchmarks</i>								
MCPBench [18]	✓	10	10	✗	Real	911	✗	Rule
MCP-RADAR [8]	✓	9	42	✓	Real	300	✗	Rule
MCP-Zero [6]	✓	308	2,797	✗	-	-	-	-
MCP Eval [15]	✓	12	77	✗	Syn.	676	✓	LLM
LiveMCPBench (ours)	✓	70	527	✓	Real	95	✓	LLM

establishes a reproducible toolset for large-scale MCP performance analysis (see Appendix F for distribution details).

2.3 LiveMCPEval

Evaluating agent trajectories in dynamic environments is non-trivial, as task outcomes often depend on evolving contexts, and multiple tool invocation trajectories may lead to equally valid solutions. Such properties make traditional metrics, such as exact tool-matching accuracy, inadequate. To address this, we introduce LiveMCPEval, an automatic evaluation framework that leverages an LLM-as-a-Judge system. Instead of relying on fixed ground-truth trajectories, LiveMCPEval directly verifies task completion by considering tool usage patterns and contextual feedback. A central component of this design is the use of *key points*—critical subtasks or intermediate conditions that must be satisfied for success—which can be either manually annotated or automatically extracted by LLMs (see Appendix H for examples). In our benchmark, all tasks are equipped with a verified set of key points to ensure reliable evaluation. Formally, given a task T , a set of key points P , an execution trajectory A (including retrievals and tool calls), and tool descriptions D , the evaluator performs binary classification to determine task success:

$$\mathcal{J} = \text{Evaluator}(T, P, A, D) \quad (1)$$

where $\mathcal{J} \in \{\text{Success}, \text{Failure}\}$. The overall benchmark metric is the *success rate* across tasks. Beyond accuracy, the framework is designed for openness and scalability: as new MCP servers and tasks emerge, LiveMCPEval enables seamless benchmark expansion without costly manual annotations, ensuring its long-term relevance in MCP ecosystems. A systematic comparison with existing benchmarks is presented in Table 1.

2.4 MCP Copilot Agent

Daily tasks are inherently dynamic, making fixed pipelines for tool use impractical. For instance, always retrieving before executing may fail when outputs must be combined across tools or when re-routing is required after errors. Agents must instead adapt to evolving contexts.

We formulate tool retrieval and invocation as a Partially Observable Markov Decision Process [28, POMDP], where decisions rely only on tool descriptions and execution feedback. The environment is defined by: (1) hidden states $\mathcal{S}$ for the underlying task; (2) observations $\mathcal{O}$ from retrieved descriptions and feedback; (3) a language action space $\mathcal{A}$; (4) transition $\mathcal{T} : \mathcal{S}_t \times \mathcal{A} \rightarrow \mathcal{S}_{t+1}$; (5) reward $\mathcal{R} : \mathcal{S} \rightarrow \mathbb{R}$ for task success. The action space $\mathcal{A}$ consists of three operations: *Route*, which retrieves k candidate tools (fixed at $k = 5$ in our experiments) from the full toolset; *Execute*, which invokes a selected tool with specified parameters and returns its output as feedback; *Response*, which terminates the process and delivers the final result to the user. Tool score in *Route* follows MCP-Zero [6]:

$$\text{score} = (s_{\text{server}} \times s_{\text{tool}}) \times \max(s_{\text{server}}, s_{\text{tool}}) \quad (2)$$

where s_{server} and s_{tool} denote cosine similarity to server-level and tool-level descriptions. This form emphasizes joint alignment while letting server priors dominate when tool matches are weak.

Our agent builds on ReACT, integrating reasoning and action in a unified loop. Unlike conventional APIs treating tools as isolated black boxes, MCP introduces a server–tool hierarchy that demands structural reasoning and coordination. The MCP Copilot Agent thus adapts flexibly under uncertainty while exploiting MCP’s hierarchical richness.

3 Experiments and Results

3.1 Setup

We evaluate 12 frontier models: Claude-Opus-4 and Claude-Sonnet-4 [2], GPT-4.1 and GPT-4.1-Mini [20], GPT-5 and GPT-5-Mini [21], Gemini-2.5-Pro [9], Deepseek-V3 and Deepseek-R1 [4], Qwen3-235B-A22B and Qwen3-32B [32], and Qwen2.5-72B-Instruct [25]. For evaluation, we employ Deepseek-V3 as default evaluator.

Tool Retrieval Configuration. To ensure consistent retrieval performance across experiments, we adopt a standardized configuration. Specifically, we employ Qwen2.5-72B-Instruct to generate server-level tool descriptions and Qwen3-Embedding-0.6B [35] to obtain dense embeddings. In all experiments, the retrieval system

Table 2: Performance and efficiency metrics: Steps denotes average dialogue turns, Tools indicates average used tools, *execute* represents average tool executions, *route* refers to average retrievals, Tokens refers to total tokens consumed (in millions), and Overall shows average task success rate.

Model	Steps	Tools	<i>execute</i>	<i>route</i>	Tokens (M)	Overall (%)
Claude-Sonnet-4-20250514	20.09	2.71	5.59	2.98	6.07	78.95
Claude-Opus-4-20250514	25.53	3.40	6.93	4.35	7.16	70.53
GPT-5	20.89	1.77	7.24	2.91	6.84	52.63
Qwen3-235B-A22B	10.33	1.24	2.11	2.00	2.88	48.42
DeepSeek-R1-0528	16.76	1.59	5.12	1.77	3.73	48.42
GPT-5-Mini	11.18	1.28	2.88	1.37	3.63	45.26
GPT-4.1-Mini	10.89	1.37	2.71	1.65	2.96	44.21
Qwen2.5-72B-Instruct	11.22	1.31	2.80	1.38	3.55	43.16
DeepSeek-V3-0324	8.33	1.01	1.29	1.41	2.11	42.11
Gemini-2.5-Pro	8.08	0.99	1.46	1.35	2.03	41.05
GPT-4.1	9.03	1.31	1.72	1.64	2.48	38.95
Qwen3-32B	9.99	1.16	2.31	1.19	1.93	30.53

returns the top-5 tools most relevant to the query, ensuring a controlled and comparable evaluation setup.

3.2 Main Results

To compare the behavioral characteristics of different models, we present the average number of dialogue turns, used tools, tool execution attempts, retrieval calls and task success rates in Table 2. Based on these metrics, we draw the following conclusions:

- (1) **Performance Variance Across Models.** We observe substantial variance in performance: while most contemporary models achieve only 30%–50% task success rates, the Claude series substantially outperforms them.
- (2) **Active tool composition correlates with stronger performance.** Claude-Sonnet-4, with 2.71 tools, 5.59 executions, and 2.98 retrievals, achieves the highest success rate, far outperforming conservative models such as Gemini-2.5-Pro or DeepSeek-V3, which use only one tool on average. This suggests that proactive interaction with the tool environment is a key mechanism for effective MCP usage.
- (3) **Efficiency hinges on balancing exploration and exploitation.** While Claude-Opus-4 explores even more aggressively (3.40 tools), its success rate drops to 70.53%, indicating diminishing returns and potential error accumulation. Conversely, models that under-explore (e.g., Qwen3-32B at 1.16 tools) fail to adapt. Thus, performance is not determined by raw tool counts but by how effectively a model manages the exploration-exploitation trade-off.

Is LiveMCPEval Reliable? We evaluated the reliability of LiveMCPEval by comparing its automatic judgments with human annotations on execution trajectories of top Claude models and by testing models as evaluators (Figure 3). Results show that LiveMCPEval is accurate under appropriate evaluators: Deepseek-V3 reaches 78.95% human agreement, while GPT-4.1 Mini and Qwen2.5-72B-Instruct achieve ~75%. In contrast, Deepseek-R1, Claude-Opus-4, and Qwen3-32B yield lower agreement (60–70%), likely due to difficulty handling long trajectories. Overall, LiveMCPEval is dependable, but its effectiveness depends on the model.

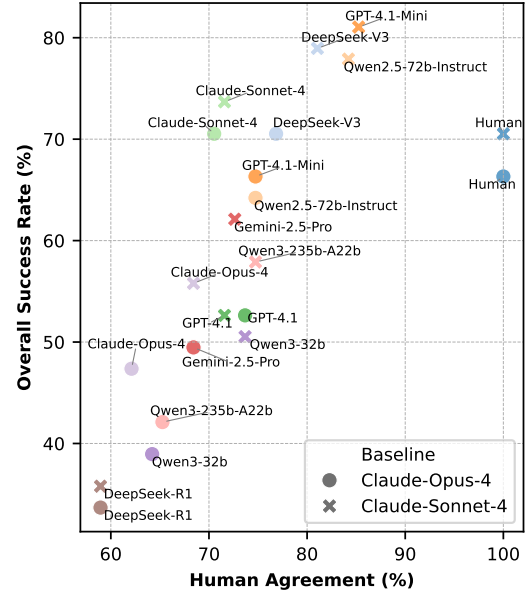


Figure 3: Correlation between evaluation performance and human agreement across evaluators on Claude trajectories.

Is LiveMCPEval Generalizable? We tested the generalizability of LiveMCPEval on Claude-Sonnet-4’s trajectories by examining how LLM-generated key points affect human agreement (Figure 4, examples in Appendix H). Results show that LiveMCPEval generalizes well across models: even without human-annotated references, most evaluators achieved higher agreement using automatically generated key points. This demonstrates the framework’s openness to new tasks and tools, since auto-generated key points can adaptively serve as substitutes when human references are unavailable. Moreover, Deepseek-V3 best leverages human-annotated key points, suggesting advantages for scenarios where such references are available.

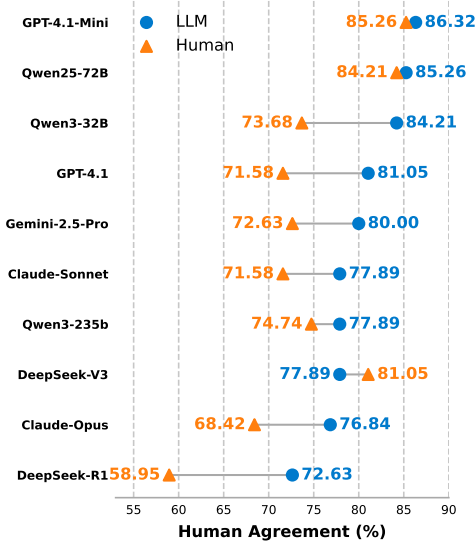


Figure 4: Human agreement rates of different evaluators when using manual versus LLM-generated key points.

4 Analysis

4.1 Efficiency Analysis

In practical applications, a trade-off between model performance and cost must be carefully considered.

Table 2 reports the average number of steps and total token consumption for each model. As expected, token usage increases with the steps. However, we observe that Claude models and GPT-5 consume substantially more tokens (6–7M) compared to the other models (2–3M). Beyond their higher average step counts, their more aggressive tool-use behavior also contributes to the increased token usage. For example, Claude-Sonnet-4 requires an average of 20.09 steps and consumes 6.70M tokens, whereas Qwen3-235B requires 16.76 steps and consumes only 3.73M tokens. This large gap is largely explained by their tool-use patterns: Claude-Sonnet-4 invokes an average of 2.71 tools per task, while Qwen3-235B uses only 1.59. These findings suggest that more aggressive tool exploration leads to substantially higher token consumption, due to factors such as more exploratory trials and error-recovery attempts.

To provide actionable insights for model selection, we plotted the relationship between logarithmic cost and performance, along with the corresponding Pareto frontier [16]. As illustrated in Figure 5. The models positioned on the Pareto frontier represent the most cost-effective choices. These models demonstrate distinct advantages in terms of cost-performance efficiency.

4.2 Impact of Tool Retrieval Configuration

To assess the impact of retriever configurations on the agent, we conduct ablation experiments on two factors: the number of retrieved tools and the choice of embedding model. Results are summarized in Table 3. For all paired comparisons against the main setting, we apply an exact McNemar test [19] on per-instance success outcomes, which is appropriate for our matched binary predictions

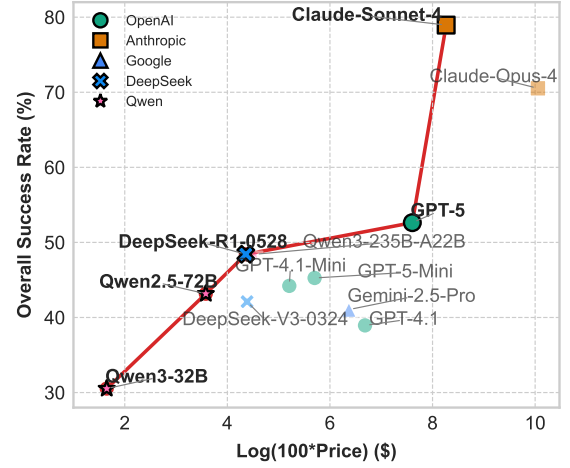


Figure 5: Log-Price vs. Performance scatter plot with Pareto frontier. Colors represent model families.

and avoids asymptotic assumptions. The resulting p -values provide evidence for the robustness of our primary hyperparameter choice.

Regarding k , reducing the number of retrieved tools from our default setting of 5 to 1 leads to a performance degradation, with the success rate dropping from 78.95% to 64.21%. This decline is statistically significant under the exact McNemar test ($p = 0.02 < 0.05$). In contrast, increasing k further to 10 yields no additional gain, as the success rate plateaus at 78.95% ($p = 1.00$). This suggests that our primary configuration is already near-optimal. We attribute this saturation to the retriever’s limited ability to surface additional feasible tools, rather than a lack of candidates in the pool.

For the embedding model, substituting Qwen3-Embedding-0.6B with BGE-M3 [3] leads to only a minor decrease in performance (from 78.95% to 76.84%), and the difference is not significant by the exact McNemar test ($p = 0.84$). Taken together, these statistically grounded comparisons suggest that our main experimental setting is stable to reasonable variations in both k and embedding choice, and further support the conclusion that the dominant bottleneck lies in the current tool-retrieval methodology itself rather than in a specific embedding model or hyperparameter.

4.3 Single Evaluator Bias Analysis

Although the reliability of LiveMCPEval has been preliminarily validated through human agreement rates, further in-depth analysis of its applicability and potential improvements is crucial for the long-term sustainability of the evaluation process. The potential impact of evaluator bias on the reliability of LLM-as-Judge has raised concerns about the robustness of such evaluation systems.

A potential solution is majority voting. To assess the effectiveness of this approach, we used all 10 models from the main experiment as evaluators to assess the trajectory of Claude-Sonnet-4. The default evaluator, deepseek-v3, has a human agreement rate of 81.05%, while the weakest evaluator, deepseek-r1, has a human agreement rate of 58.95%. We considered results with at least a half vote as predictions, yielding a human agreement rate of 77.89%.

Table 3: Ablation study on the number of retrieved tools (k) and embedding model selection. We employ Claude-Sonnet-4 as the agent and DeepSeek-V3 as the evaluator.

Settings	Overall (%)	McNemar p-value
Qwen3-Embedding-0.6B + $k=5$ (main)	78.95	-
<i>Change k</i>		
Qwen3-Embedding-0.6B + $k=1$	64.21	0.02
Qwen3-Embedding-0.6B + $k=10$	78.95	1.00
<i>Change Embedding Model</i>		
BGE-M3 + $k=5$	76.84	0.84

This rate is slightly lower than that of the default evaluator but significantly higher than the weakest evaluator. To further explore the upper performance limit of multiple models, we calculated the maximum human agreement rate (where any model output agreeing with human results is considered correct) to be **97.89%**. These findings suggest that when the performance of individual evaluators is uncertain, using a multi-model voting approach is a more robust choice.

To further verify whether the specific model bias of our default deepseek-v3 evaluator affects the relative ranking of performance, we also used **Qwen2.5-72B-Instruct as an evaluator** and reassessed all trajectories. The experimental results, shown in Table 4, indicate that the relative rankings remained almost unchanged.

To quantitatively examine the consistency between the two evaluator, we further computed Kendall’s τ -b [13], which measures the ordinal association between two ranked lists while accounting for ties. The result (τ -b = **0.8837** p = **0.00008**) indicates a strong and statistically significant agreement between the rankings. In other words, although individual evaluators may exhibit model-specific preferences, their induced orderings over trajectories are largely aligned.

4.4 Evaluation Error Analysis

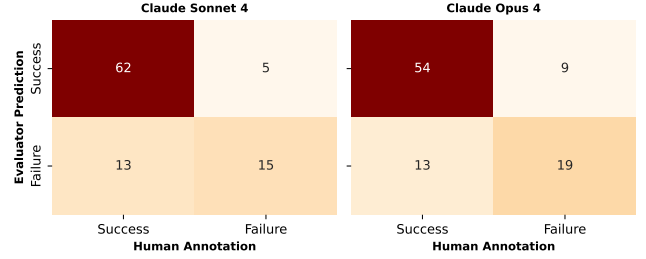
Figure 6 presents the confusion matrices between the model-based evaluator (DeepSeek-v3) and human annotators for the Claude family of models. The evaluation of claude-sonnet-4 achieved an F1 score of 87.32, while claude-opus-4 reached 83.08, underscoring the relatively high reliability of model-based evaluation. To better understand the observed discrepancies between human judgments and automatic evaluation, we conducted a detailed error analysis.

For clarity and mutual exclusiveness, we categorize evaluation errors into the following four types:

- **Output Completeness Error (30%):** Cases where the evaluator disagreed with humans on whether the model output provided a fully correct and final solution, versus only partial or intermediate steps.
- **Hallucinated Completion (45%):** Cases where the agent incorrectly assumed completion of a task or access to unavailable information, leading the evaluator to mistakenly align with this hallucination.

Table 4: Reevaluation of main experiment results using Qwen2.5-72B-Instruct. The relative rankings exhibit strong consistency with the original evaluation (Kendall’s τ -b = 0.8837, $p < 0.001$).

Model	Overall (%)	Origin Rank	New Rank
Claude-Sonnet-4	77.89	1	1
Claude-Opus-4	64.21	2	2
GPT-5	51.58	3	3
Qwen3-235B	40.00	4	4
GPT-5-Mini	40.00	6	5
Qwen2.5-72B	38.95	8	6
Deepseek-R1	37.89	5	7
GPT-4.1-Mini	31.58	7	8
Deepseek-V3	31.58	9	9
Gemini-2.5-Pro	29.79	10	10
GPT-4.1	27.37	11	11
Qwen3-32B	25.26	12	12

**Figure 6: Confusion matrices comparing model-based evaluation (DeepSeek-v3) against human annotators.**

- **Granularity Mismatch (17.5%):** Cases where the evaluator and humans diverged due to differences in tolerance for detail, such as penalizing superficial formatting inconsistencies or overlooking missing justifications.
- **Others (7.5%):** Residual errors, primarily attributable to human annotation mistakes rather than systematic evaluator weaknesses.

4.5 Error Analysis

We conducted a detailed error analysis on the trajectories of current retrieval and invocation agents to provide insights for future development. Human annotators were employed to classify error types in the trajectories of Claude-Opus-4 and Claude-Sonnet-4. Based on the modules in the MCP Copilot Agent framework, we identified four distinct error categories (Figure 7). Each erroneous trajectory was uniquely classified into one error type without overlap. Detailed error examples are provided in Appendix I.

Query Error. Query errors occur when the generated query either lacks semantic relevance to the required tools or exhibits a granularity mismatch with tool capabilities. For instance, in the task “summarize today’s news and save as PDF,” the agent might

request a single omnipotent tool despite the availability of specialized tools for news retrieval and PDF generation. Such granularity mismatches prevent the retrieval system from providing appropriate tools, and agents often fail to refine queries based on retrieval feedback. Hallucinated queries for irrelevant tools further exacerbate this issue. These errors stem from limitations in LLMs’ task decomposition and planning capabilities, suggesting room for improvement despite their generally competent performance.

Retrieve Error. Retrieve errors arise when semantically appropriate queries fail to match available tools due to retrieval system shortcomings. For example, in the task “Convert the YouTube video to MP3 format,” the retrieval system may overlook the *youtube downloader* tool (which supports format conversion) due to unrecognized semantic equivalence between “convert to MP3” and the tool’s documented “extract audio tracks” functionality. These errors highlight challenges in hierarchical retrieval (e.g., MCP server-tool structures) and semantic similarity computation. Dominating the error distribution, retrieval errors underscore the critical need for enhanced retrieval architectures and more robust similarity metrics.

Tool Error. Tool errors occur when the agent retrieves the correct tool but invokes it incorrectly—e.g., via error parameters or incomplete server/tool names. In the task “summarize news and save to specified path,” the agent might supply “path name” instead of the required “path” parameter to the save tool. Such inaccuracies reflect limitations in contextual precision and memory retention. While modern LLMs exhibit strong contextual understanding, these errors indicate a need for more sophisticated memory mechanisms to ensure reliable tool usage.

Other Error. This category encompasses sporadic failures beyond the above types, including network timeouts or model invocation errors. For example, in “summarize today’s news,” a network timeout during news retrieval may cause the agent to abandon the task without retries or alternative solutions. Such behavior reveals deficiencies in framework design, particularly the absence of robust error-handling mechanisms (e.g., failure recovery, adaptive tool exploration). The prevalence of these errors suggests that while current frameworks support basic exploration, significant improvements in fault tolerance and proactive problem-solving are needed.

5 Related Work

5.1 Tool-Use Benchmarks

Most existing benchmarks for tool use rely on simulated APIs, given the instability of real-world interfaces. For example, API-Bank [14] and τ -bench [33] construct artificial toolsets to ensure stability, while ToolAlpaca [29] and Seal-Tools [31] collect real-world APIs but cannot execute actual calls. A third line of work, such as ToolBench [22] and ShortcutsBench [27], attempts to integrate real APIs, but frequent interface changes often render tools unusable [11]. More recent efforts, e.g., StableToolBench-MirrorAPI [10], use fine-tuned LLMs to simulate APIs and calls. Yet, these benchmarks remain largely API-centric, inheriting instability and lacking support for broader functionality—such as direct manipulation of local files. The emergence of MCP offers a new paradigm: a stable, unified

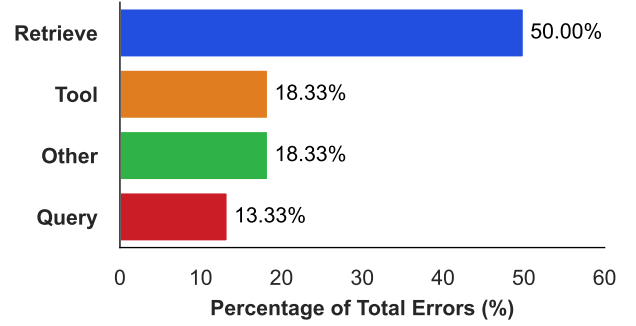


Figure 7: Distribution of errors for four types. We counted trajectories of Claude series models.

interface for building general-purpose toolsets. In this work, we leverage MCP to construct a practical toolset that overcomes both API instability and functional limitations, enabling a comprehensive and reliable real-world tool-use benchmark.

5.2 MCP Benchmarks

The evaluation of MCP systems is still in its early stages and continues to evolve rapidly. Among existing efforts, MCPBench [18] represents one of the first benchmarks, mainly comparing MCP tools with traditional API-based tools. Building on this, MCP-RADAR [8] broadens the scope by introducing a multi-dimensional framework that evaluates efficiency, accuracy, and robustness. More recently, MCPEval [15] has advanced the field with a fine-grained framework capable of automatically generating queries to assess MCP server performance. Despite these advances, current benchmarks share a key limitation: they typically evaluate only small-scale MCP servers (around 10 servers), which fail to capture real-world settings where agents must operate in large, dynamic MCP ecosystems. To address this gap, we introduce a large-scale MCP toolset and systematically examine agent capabilities in accomplishing daily tasks through tool use. Recent work, such as RAG-MCP [7], MCPZero [6], and ScaleMCP [17] has begun exploring retrieval over large-scale MCP toolsets. However, these approaches are constrained by rigid pipelines that lack adaptability in tool invocation and error recovery. Moreover, ScaleMCP depends on a manually constructed toolset with limited functional diversity, reducing its applicability to broader real-world scenarios.

6 Conclusion

We presented LiveMCPBench, a benchmark consisting of 95 daily tasks for evaluating tool-use agents in large-scale MCP ecosystems. Alongside, we released LiveMCPTool, a ready-to-use collection of 527 tools, and proposed LiveMCPEval, an automatic evaluation framework that handles task dynamism and solution diversity. We further built the MCP Copilot Agent to study tool retrieval and composition in large-scale MCP settings. Our experiments across 12 frontier models highlight persistent limitations: tool retrieval remains the primary bottleneck, while effective tool composition proves equally critical. These findings underscore the need for

future systems that jointly advance tool retrieval and foster robust tool composition capabilities.

References

- [1] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Accessed: 2026-01-28.
- [2] Anthropic. 2025. Introducing Claude 4. <https://www.anthropic.com/news/claude-4>. Accessed: 2026-01-28.
- [3] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2025. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL] <https://arxiv.org/abs/2402.03216>
- [4] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [5] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). arXiv:2505.02279 [cs.AI] <https://arxiv.org/abs/2505.02279>
- [6] Xiang Fei, Xiaowu Zheng, and Hao Feng. 2025. MCP-Zero: Active Tool Discovery for Autonomous LLM Agents. arXiv:2506.01056 [cs.AI] <https://arxiv.org/abs/2506.01056>
- [7] Tianfeng Gan and Qiyao Sun. 2025. RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation. arXiv:2505.03275 [cs.AI] <https://arxiv.org/abs/2505.03275>
- [8] Xuanqi Gao, Siyi Xie, Juan Zhai, Shiqing Ma, and Chao Shen. 2025. MCP-RADAR: A Multi-Dimensional Benchmark for Evaluating Tool Use Capabilities in Large Language Models. arXiv:2505.16700 [cs.AI] <https://arxiv.org/abs/2505.16700>
- [9] Google. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs.CL] <https://arxiv.org/abs/2507.06261>
- [10] Zhicheng Guo, Sijie Cheng, Yuchen Niu, Hao Wang, Sicheng Zhou, Wenbing Huang, and Yang Liu. 2025. StableToolBench-MirrorAPI: Modeling Tool Environments as Mirrors of 7,000+ Real-World APIs. arXiv:2503.20527 [cs.CL] <https://arxiv.org/abs/2503.20527>
- [11] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 11143–11156. doi:10.18653/v1/2024.findings-acl.664
- [12] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278 [cs.CR] <https://arxiv.org/abs/2503.23278>
- [13] Maurice George Kendall. 1948. Rank correlation methods. (1948).
- [14] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *The 2023 Conference on Empirical Methods in Natural Language Processing*. <https://openreview.net/forum?id=o2HBfgY20b>
- [15] Zhiwei Liu, Jielin Qiu, Shiyu Wang, Jianguo Zhang, Zuxin Liu, Roshan Ram, Haolin Chen, Weiran Yao, Huan Wang, Shelby Heinecke, Silvio Savarese, and Caiming Xiong. 2025. MCP Eval: Automatic MCP-based Deep Evaluation for AI Agent Models. arXiv:2507.12806 [cs.AI] <https://arxiv.org/abs/2507.12806>
- [16] Alexander V Lotov, Vladimir A Bushenkov, and Georgy K Kamenev. 2004. *Interactive decision maps: Approximation and visualization of Pareto frontier*. Vol. 89. Springer Science & Business Media.
- [17] Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honagana-halli Basavaraju, and James A. Burke. 2025. ScaleMCP: Dynamic and Auto-Synchronizing Model Context Protocol Tools for LLM Agents. arXiv:2505.06416 [cs.CL] <https://arxiv.org/abs/2505.06416>
- [18] Zhiling Luo, Xiaorong Shi, Xuanrui Lin, and Jinyang Gao. 2025. Evaluation Report on MCP Servers. arXiv:2504.11094 [cs.LR] <https://arxiv.org/abs/2504.11094>
- [19] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996
- [20] Openai. 2025. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/>. Accessed: 2026-01-28.
- [21] Openai. 2025. Introducing GPT-5. <https://openai.com/gpt-5/>. Accessed: 2026-01-28.
- [22] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations* <https://openreview.net/forum?id=dHng2O0Jjr>
- [23] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2025. Tool learning with large language models: A survey. *Frontiers of Computer Science* 19, 8 (2025), 198343.
- [24] Qwen. 2025. Qwen3: Think Deeper, Act Faster. <https://qwenlm.github.io/blog/qwen3/>. Accessed: 2026-01-28.
- [25] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuhong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
- [26] Partha Pratim Ray. 2025. A survey on model context protocol: Architecture, state-of-the-art, challenges and future directions. *Authorea Preprints* (2025).
- [27] Haiyang SHEN, Yue Li, Desong Meng, Dongqi Cai, Sheng Qi, Li Zhang, Mengwei Xu, and Yun Ma. 2025. ShortcutsBench: A Large-Scale Real-world Benchmark for API-based Agents. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=kKILfPkhSz>
- [28] David Silver and Joel Veness. 2010. Monte-Carlo Planning in Large POMDPs. In *Advances in Neural Information Processing Systems*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta (Eds.), Vol. 23. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2010/file/edfbc1afcf9246bbd40eb4d8027d90f-Paper.pdf
- [29] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. 2023. ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases. arXiv:2306.05301 [cs.CL] <https://arxiv.org/abs/2306.05301>
- [30] Lei Wang, Chen Ma, Xuexiang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [31] Mengsong Wu, Tong Zhu, Han Han, Chuanyuan Tan, Xiang Zhang, and Wenliang Chen. 2025. Seal-Tools: Self-instruct Tool Learning Dataset for Agent Tuning and Detailed Benchmark. In *Natural Language Processing and Chinese Computing*, Derek F. Wong, Zhongyu Wei, and Muyun Yang (Eds.). Springer Nature Singapore, Singapore, 372–384.
- [32] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuhong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [33] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. 2025. $\{ \tau \}$ -bench: A Benchmark for $\{ \tau \}$ -agent- $\{ \tau \}$ Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=roNSXZpUDN>
- [34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X
- [35] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. arXiv:2506.05176 [cs.CL] <https://arxiv.org/abs/2506.05176>
- [36] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 46595–46623. https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf

A Limitations

While LiveMCPBench represents a comprehensive benchmarking framework, we acknowledge several limitations in its design and evaluation methodology:

Dependence on LLM evaluation. The LiveMCPEval component relies heavily on LLM-based evaluation. Although we have validated the accuracy through human experiments, potential model biases may still influence the results. To mitigate this concern, we conducted case studies analyzing model judgment failure cases, which helps improve the robustness of our evaluation framework.

Evaluation Assumptions. Our assessment framework operates under the assumption that agent behavior trajectories and tool descriptions sufficiently reflect task performance, without explicitly verifying the final environmental impact. While this assumption holds in most cases, expanding the toolset could introduce inconsistencies between actual tool effects and their descriptions, potentially compromising evaluation reliability. To address this, we rigorously inspect the quality of LiveMCPTool to minimize such discrepancies.

Server Reliability and Maintenance. While our framework highlights the “plug-and-play” nature of LiveMcpTool, we acknowledge that the current work does not explicitly address long-term aspects such as security, versioning, and maintenance of deployed servers. In practice, sustainable applicability requires systematic guidelines for server updates, monitoring, and health curation. To ensure reproducibility of our reported results, we provide a fixed and validated toolset version that has undergone security checks, which guarantees a safe and consistent experimental environment. Beyond this controlled release, we view long-term sustainability as an orthogonal but essential research direction: future iterations of LiveMCPBench could incorporate auditing modules or best-practice recommendations (e.g., periodic health checks, semantic versioning, and community-curated registries) to further ensure secure and reliable deployment.

B Ethical Considerations

The advent of large-scale multi-tool retrieving and calling agents promises to revolutionize traditional UI-based interaction paradigms by shifting from complex message retrieval or manual UI operations to automated tool invocation. This transition holds significant potential to reduce usability barriers, enhance operational efficiency, and accelerate progress toward Artificial General Intelligence (AGI). Furthermore, such systems can augment the capabilities of smaller models through automated tool construction by larger models. For instance, when faced with tasks beyond their native competence (e.g., complex code generation), smaller models can leverage tools dynamically encapsulated by larger models through MCP interfaces.

However, alongside these benefits, our framework introduces potential risks that warrant careful consideration. Malicious actors could exploit the system by disguising harmful or unsafe tools through misleading descriptions, potentially inducing models to execute dangerous operations. Such misuse may lead to information security breaches or financial losses. Additionally, erroneous tool

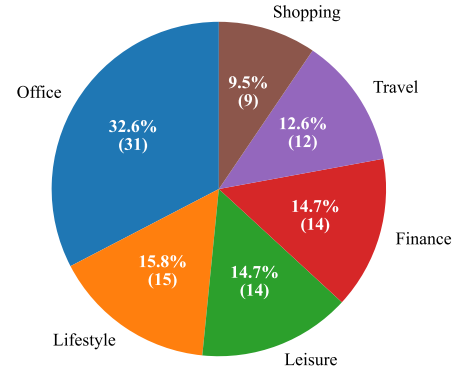


Figure 8: Task distribution in LiveMCPBench: a benchmark comprising 95 daily tasks across 6 distinct domains.

invocation by the model—such as unintended deletion of local files—could cause significant losses, underscoring the need for robust safeguards in tool validation and execution monitoring.

C LLM Usage

We used LLMs, specifically OpenAI’s GPT-based tools, only to check grammar, improve readability, and polish draft wording. All conceptual, technical, and scientific contributions are entirely the authors’ own.

D Reproducibility statement

We open-source all the code and human annotations used in our experiments. In addition, we include the complete set of tasks and the entire LiveMCPTool. Furthermore, Appendix G details the computational resources we utilized, the access methods for private model APIs.

E Details of Task Construction

E.1 Task Statistics

The task statistics of LiveMCPBench are illustrated in Figure 8. LiveMCPBench comprises six categories of tasks, each designed to reflect common real-life scenarios:

- (1) **Office.** This category represents typical office-related tasks, primarily involving reading and writing documents in Word, Excel, and PowerPoint.
- (2) **Lifestyle.** These tasks pertain to daily routines, such as retrieving news updates or querying the latest arXiv papers.
- (3) **Leisure.** This category encompasses entertainment-oriented tasks, including fetching gaming news, obtaining specific game-related information, or retrieving details about museums.
- (4) **Finance.** Tasks in this category focus on personal financial management, such as checking stock prices, analyzing market trends, or obtaining cryptocurrency valuations.
- (5) **Travel.** This category includes tasks related to personal travel, such as route planning, hotel searches, and ticket inquiries.

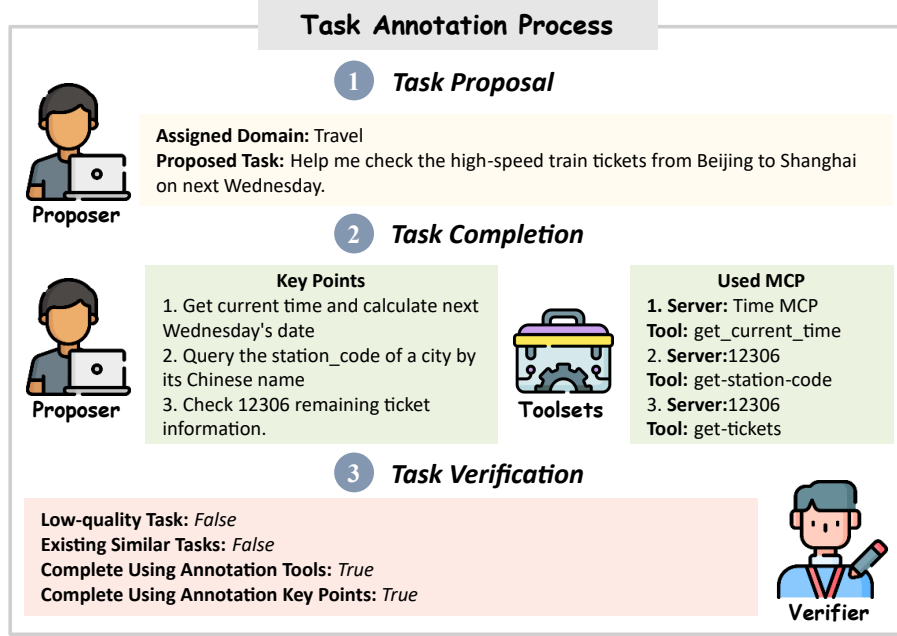


Figure 9: Example of the task annotation process.

- (6) **Shopping.** These tasks revolve around personal shopping activities, including product information retrieval and recommendations.

Importantly, the retained 95 tasks were selected not for sheer quantity, but for domain coverage, generality, and compositional complexity. For instance, beyond single-step retrieval, many tasks involve multi-hop reasoning (e.g., analyzing market data followed by producing a spreadsheet report) or cross-domain integration (e.g., combining travel planning with lifestyle queries). Compared to prior agent benchmarks, LiveMCPBench thus prioritizes both breadth of domains and depth of task composition, providing a balanced yet realistic testbed despite the smaller absolute number of tasks relative to very large-scale static datasets.

Annotator MCP Usage. To accomplish the benchmark tasks, annotators utilized 55 out of the 70 available servers (78.57%) and 150 out of the 527 available tools (28.46%). This indicates that while not all resources were required, a substantial and diverse subset of servers and tools was actively engaged, reflecting realistic large-scale MCP tool usage.

In practice, many MCP servers contain noisy or redundant tools that are not essential for task completion, and our benchmark naturally captures this characteristic by relying on only the most relevant subset.

E.2 Annotation Principles

LiveMCPBench focuses on leveraging a large-scale MCP toolset to accomplish complex tasks. To ensure high-quality task construction, we employ two groups of annotators involving *proposers* and *verifiers* (see Figure 9). All annotators first freely explore the MCP

toolset, including tool descriptions and real-world calls, to gain familiarity with its functionalities.

First, *Proposers* are randomly assigned a scenario and instructed to formulate tasks adhering to the following principles:

- (1) **Real-World Relevance.** Tasks must reflect realistic needs within the given scenario.
- (2) **Temporal Dynamics.** Tasks should be time-sensitive, requiring real-time information retrieval from tools rather than relying solely on static internal knowledge.
- (3) **Tool Diversity.** Tasks should necessitate the integration of multiple tools, avoiding cases where a single tool suffices for completion.

After proposing a task, the *proposers* try to complete it using the MCP toolset, documenting the required tools and key points.

Once all tasks are collected, *verifiers* manually consolidate similar tasks to prevent redundancy. Additionally, they rigorously assess task feasibility and execution quality to maintain high standards in the benchmark.

F Details of LiveMCPTool Collection

F.1 Toolset Statistics

The statistics of LiveMCPTool’s tools and servers are illustrated in Figures 10–11. The collected toolset is categorized into eight distinct classes:

- (1) **Discovery.** This category encompasses tools for information gathering and retrieval, such as search engines and news aggregators.

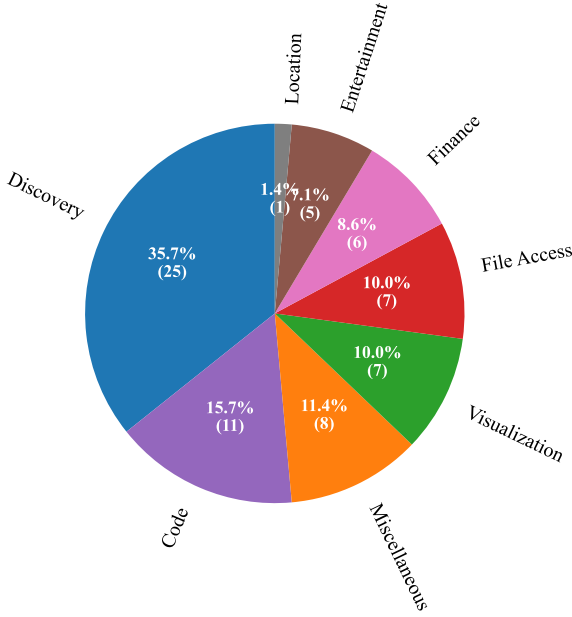


Figure 10: Distribution of servers in LiveMCPTool, categorized into 8 distinct types (70 servers).

- (2) **Visualization.** Tools in this category facilitate data or concept visualization, including bar chart plotting and mind map creation.
- (3) **File Access.** This class comprises tools for local file operations, such as reading Word, Excel, or PowerPoint files, as well as executing command-line instructions.
- (4) **Code.** These are programming-related tools, such as those providing the latest AntV documentation and sample code.
- (5) **Entertainment.** This category includes recreational tools, such as those for retrieving Yu-Gi-Oh card information or League of Legends game data.
- (6) **Finance.** Financial tools fall under this class, including those for fetching real-time stock prices or cryptocurrency market data.
- (7) **Location.** This category consists of map-based services, such as navigation systems and points-of-interest discovery tools.
- (8) **Miscellaneous.** This catch-all category accommodates tools not fitting the above classifications, such as calculators and local memory utilities.

F.2 Tool Verification for LiveMCPTool.

To address concerns regarding the dynamism of the LiveMCPTool, we conducted a systematic verification of all collected tool implementations. For tools that require online information (e.g., weather, news, or stock data), we confirmed that they indeed retrieve real data from online data sources rather than relying on simulated responses or pre-cached static datasets. This validation ensures that the server behaviors used in our benchmark faithfully reflect dynamic, real-world conditions, thereby supporting our claim of genuine runtime dynamism.

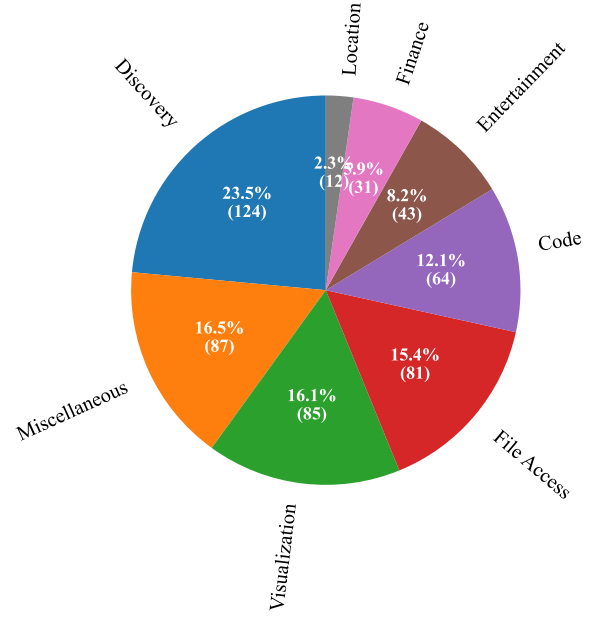


Figure 11: Distribution of tools in LiveMCPTool, categorized into 8 distinct types (527 tools).

G Implementation Details

In our experiments, we deployed two models: Qwen2.5-72B-Instruct and Qwen3-Embedding-0.6B. The computational infrastructure consisted of a Linux server (Ubuntu 22.04) with 4 NVIDIA A800-80G GPUs and 1TB of memory.

We accessed the following proprietary models through their respective platforms:

- **OpenRouter:** GPT-5, GPT-5-Mini, GPT-4.1, GPT-4.1-Mini, DeepSeek-R1-0528, DeepSeek-V3-0324, Qwen3-235B-A22B and Qwen3-32B.
- **Anthropic Console:** Claude-Opus-4-20250514, Claude-Sonnet-4-20250514.
- **Google AI Studio:** Gemini-2.5-Pro.

To address suboptimal greedy decoding in certain reasoning models, we implemented a uniform temperature parameter of 0.7 across all experiments. This configuration introduces controlled stochasticity while maintaining result reliability for long-horizon tasks, as we observed that sporadic randomness has a negligible cumulative impact on aggregate performance. We compute the total expenditure based on the model prices provided by OpenRouter as of January 31, 2026, accounting for both input and output tokens.

H Evaluation Analysis

H.1 Evaluation Metrics and Judge Model

In LiveMCPBench, we primarily adopt the single-run success rate as the main comparison metric. However, depending on different evaluation needs, alternative metrics may be more appropriate. For example, the $pass@k$ metric (success in at least one out of k independent samples) can be used to assess an agent’s ability to discover

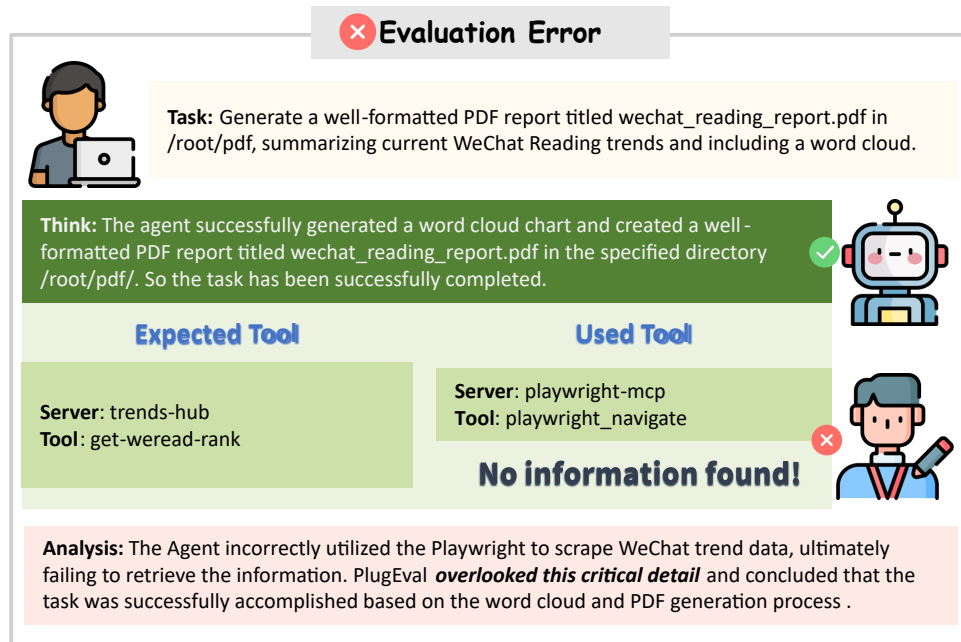


Figure 12: An illustrative case of evaluation failure in LiveMCPEval. The evaluator model erroneously concluded the task was successful based solely on the agent’s file creation action, while failing to recognize that the agent did not actually acquire the required information.

diverse solution paths. Conversely, the $\text{pass}@k$ metric (success in all k independent samples) evaluates the agent’s consistency and reliability.

For the judge model used in LIVEMCPEVAL, we recommend employing DeepSeek-V3-0324 to obtain results comparable to those reported in this paper. Nevertheless, other models with higher human agreement or more robust evaluation frameworks may also be used, particularly by leveraging our annotated trajectories based on the Claude family.

H.2 Tool-Use Hallucination

Tool-use hallucination is a major source of evaluation failure. Because the evaluator judges task completion solely from the agent’s trajectory, hallucinated tool calls or fabricated tool outputs can mislead the evaluator. Therefore, a careful analysis and practical mitigation strategy are necessary.

Conventional rule-based, post-hoc verification is difficult to scale in a large and dynamic MCP environment. Different tool combinations (e.g., multiple search engines in the MCP ecosystem) can achieve equivalent functionality, making it hard to encode capability-equivalence checks with fixed rules. Likewise, answer-based verification is brittle under dynamic tasks such as daily news summarization. As a result, reliable evaluation protocols that are robust to hallucinations are challenging to deploy in real MCP settings.

A promising alternative is to have the evaluator decompose the task into explicit completion criteria and then locate supporting evidence directly from the raw tool outputs. Tool-use hallucinations

most often arise when the agent invents intermediate information across multi-step reasoning, and the evaluator fails to notice the fabrication. Evidence-grounded, criterion-wise evaluation can substantially reduce this risk. This can be viewed as an extension of our key-points-based evaluation: each key point is verified independently against the tool sequence.

To quantify the prevalence of tool-use hallucination, we analyzed 814 tool invocations produced by Claude-Sonnet-4. Only **9.00%** involved hallucination, indicating that such errors are relatively infrequent. To assess their impact on evaluation, we measured disagreement between the DeepSeek-V3 evaluator and human judgments due to tool-use hallucination across Claude-family models. The inconsistency rate was only **1.6%**. Among 11 observed hallucination-induced errors, 8 were detected by the evaluator, yielding a **72.7%** detection rate. Thus, the current protocol already catches most such failures, and the remaining undetected cases place a very small upper bound on evaluation.

Finally, we tested whether tool-use hallucinations are reliably detected across evaluators. We compared the distribution of evaluation errors when using **Qwen2.5-72B** as the evaluator against the default DeepSeek-V3. If detection were unstable, the error-type distribution would shift substantially. Instead, the distribution remained similar: Completeness Error (33.3%), Hallucinated Completion (46.2%), Granularity Mismatch (15.4%), and Others (5.1%). This close match suggests that hallucination-related failures are captured consistently rather than introducing evaluator-dependent bias. Therefore, tool-use hallucinations are unlikely to materially distort our evaluation results.

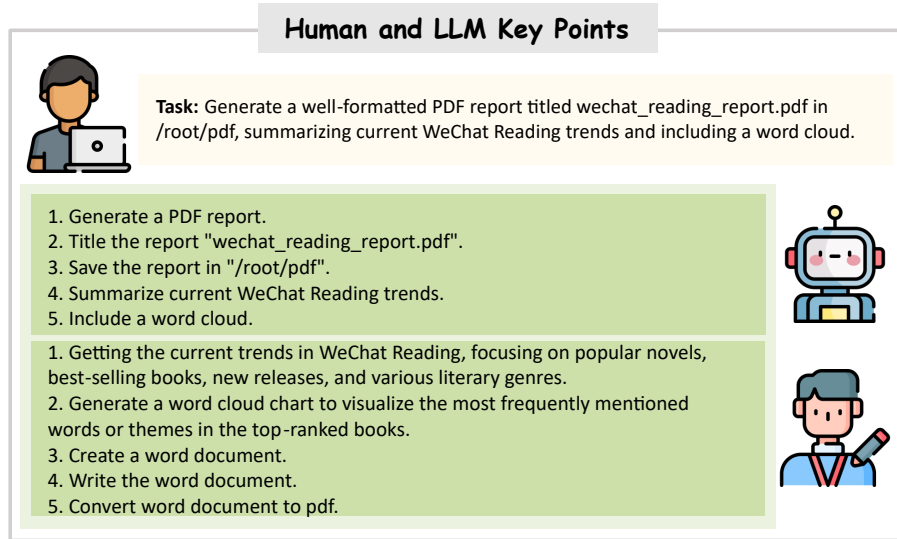


Figure 13: Comparison of key points in DeepSeek-V3 and human annotations: Similar content despite different ordering.

H.3 Case Study: Error Evaluation Examples

To illustrate cases where evaluator judgments diverge from human assessments, we conducted a case study on Claude-Sonnet-4 trajectories evaluated by DeepSeek-V3, presenting a representative example in Figure 12.

In this instance, the evaluator failed to recognize that the agent did not actually acquire the correct information, despite successfully creating the required file. The evaluator erroneously concluded task completion based solely on the file creation process. This case highlights a potential limitation of LiveMCPEval: the system’s tendency to overlook critical details when processing excessively lengthy and complex trajectories.

We propose that this long-range evaluation challenge could be addressed by modifying existing evaluation frameworks to incorporate dynamic agent-based assessment of each trajectory step. However, such an approach would significantly compromise evaluation efficiency. While our current evaluation method achieves satisfactory human agreement rates, this particular issue warrants further in-depth investigation.

H.4 Human Key Points

What constitutes a key point. A *key point* is an actionable, minimal step that contributes to solving the task using the MCP toolset. Each key point must (i) be **atomic** (one tool-mediated action per point), (ii) be **anchored** to a concrete tool or observable signal, (iii) be **auditable** from logs (inputs/outputs). Proposers write key points while completing the task, following the principles in Section E.

Normalization and consolidation. After collection, *verifiers* normalize key points into a canonical set by enforcing: (1) imperative voice; (2) one action per point; (3) explicit dependencies (e.g., “after obtaining X from Tool-A, call Tool-B with Y”); (4) de-duplication of paraphrases and overlapping sub-steps. When multiple proposers describe similar operations with different granularity,

verifiers keep the coarsest level that remains auditable and does not obscure required tool interactions.

H.5 Case Study: Human and LLM Key Points Examples

To analyze the differences between LLM-generated key points and human-annotated key points, we conducted a comparative study between key points generated by Deepseek-V3 and those manually annotated by humans. The comparison results are presented in Figure 13.

Our analysis reveals that while the ordering of key points differs between human and LLM-generated outputs, both consistently capture similar critical steps. This observation suggests the practical applicability of LLM-generated key points in evaluation tasks. Importantly, our findings indicate that LLM-generated key points can serve as a reliable alternative for robust evaluation in scenarios where human annotations are unavailable.

I Case Study: Error Examples

Figures 14–17 present concrete examples of four distinct error types: *Query Error* (13.33%), *Retrieve Error* (50.00%), *Tool Error* (18.33%), and *Other Error* (18.33%).

Broadly speaking, *Query* and *Other* errors primarily highlight design flaws in the agent’s architecture—specifically, whether the agent incorporates sufficient mechanisms to ensure task completion. In contrast, *Tool* errors are more closely tied to the capabilities of the LLM itself, particularly its ability to accurately process tool parameters and descriptions while maintaining nuanced contextual understanding. *Retrieve* errors, on the other hand, largely reflect the limitations of the tool retrieval system, testing its effectiveness in identifying relevant tools based on the server-tool description.

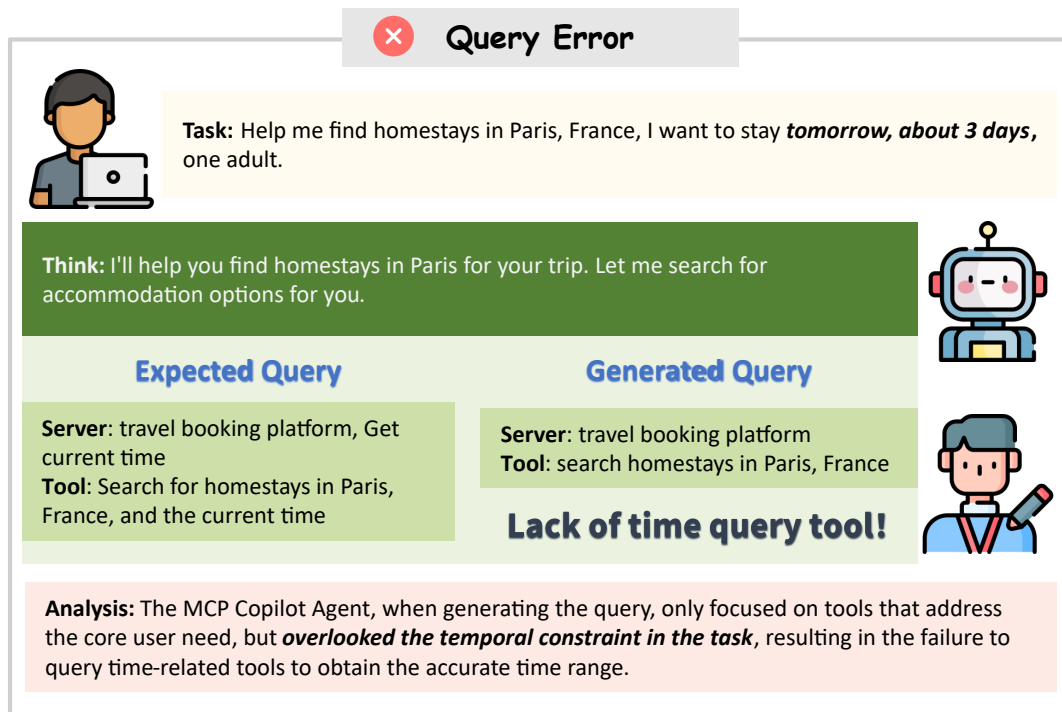


Figure 14: An illustration of Query Error: Discrepancy between the agent-generated query and the task's required competencies.

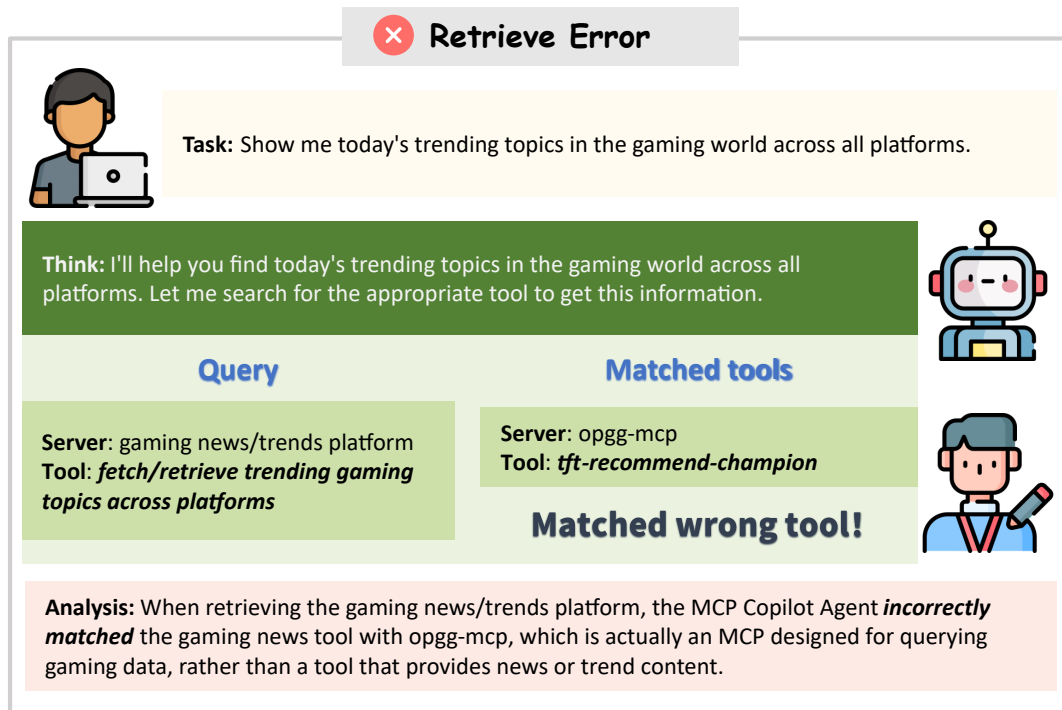


Figure 15: An illustration of Retrieve Error: The retrieve system incorrectly identifies and returns an inappropriate tool.

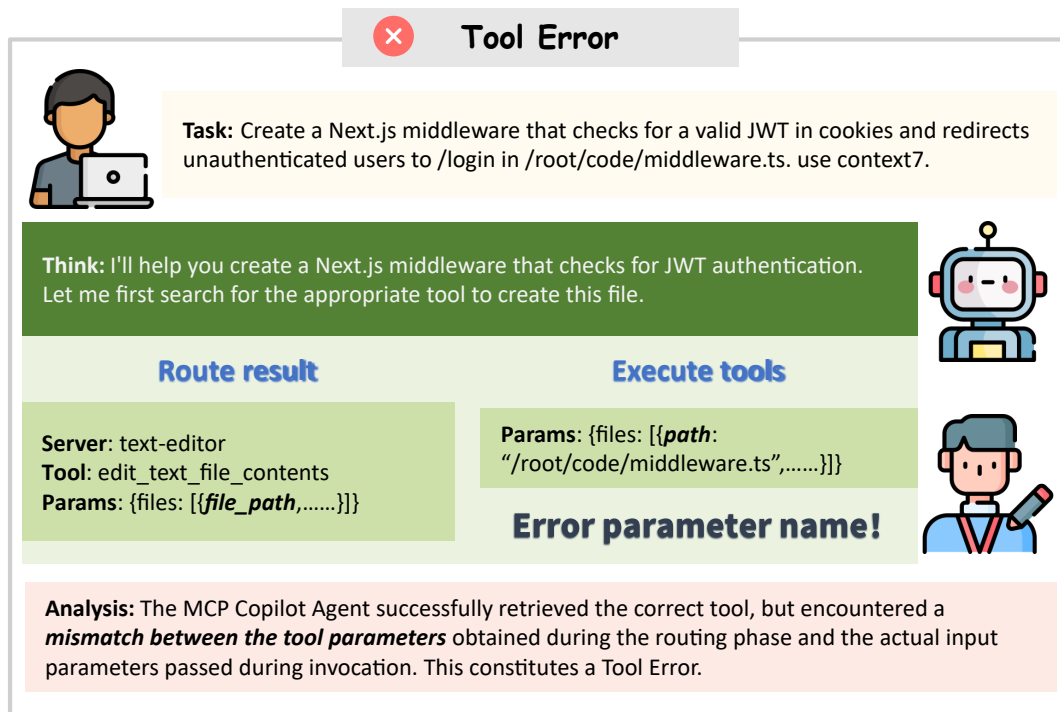


Figure 16: An illustration of Tool Error: The LLM misapplication with improper tool parameters.

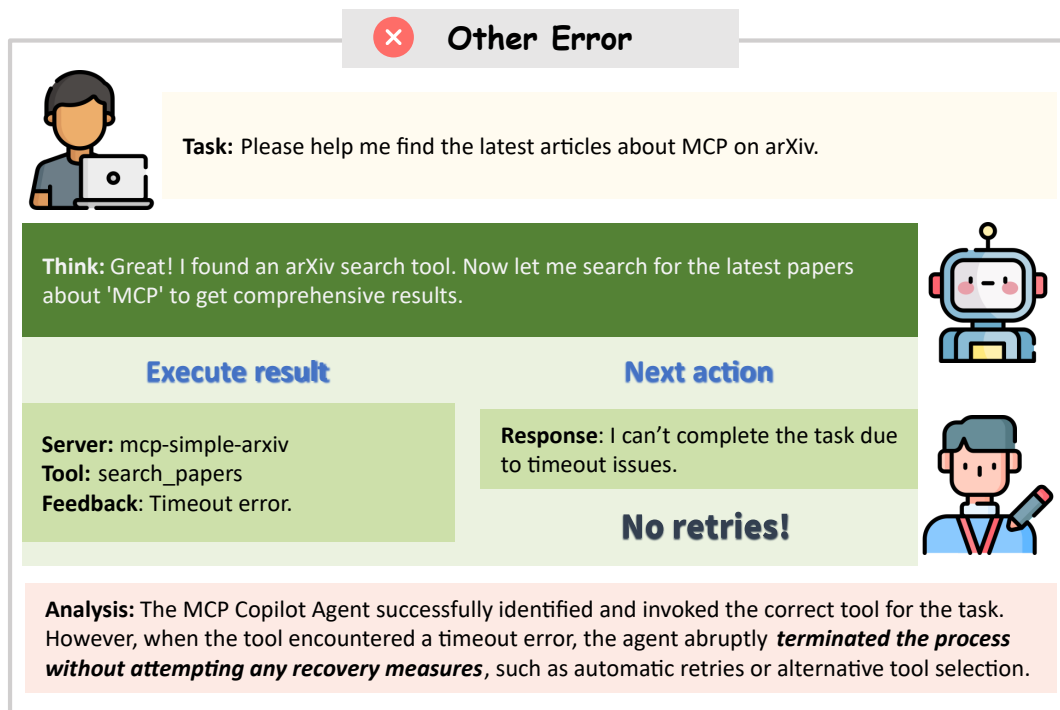


Figure 17: An illustration of Other Error: The agent's inadequate response to tool timeout.

J Prompts

Prompt for MCP Copilot Agent

You are an agent designed to assist users with daily tasks by using external tools. You have access to two tools: a retrieval tool and an execution tool. The retrieval tool allows you to search a large toolset for relevant tools, and the execution tool lets you invoke the tools you retrieved. Whenever possible, you should use these tools to get accurate, up-to-date information and to perform file operations. Note that you can only response to user once, so you should try to provide a complete answer in your response.

Task

Tool: mcp-copilot (with *route* and *excute* tool)

Prompt for *route* tool

This is a tool used to find MCP servers and tools that can solve user needs When to use this tool:

-When faced with user needs, you (LLM) are unable to solve them on your own and do not have the tools to solve the problem.

-When a user proposes a new task and you (LLM) are unsure which specific tool to use to complete it.

-When the user's request is vague or complex, and feasible tool options need to be explored first.

-This is the first step in executing unknown tasks, known as the "discovery" phase, aimed at finding the correct tool.

****Parameter Description****

Query (string, required): The input query must contain a <tool_assistant> tag with server and tool descriptions, for example:

<tool_assistant>

server: ... # Platform/permission domain

tool: ... # Operation type + target

</tool_assistant>

Prompt for server summary

You are an expert AI technical writer. Based on the following information about an MCP server, please generate a concise and accurate summary of its core purpose and capabilities.

****Server Name:**** server_name

****Server Description:**** server_desc

****Available Tools:**** tool_descriptions

Please return only the generated summary text, without any additional titles or preambles.

Prompt for *excute* tool

A tool for executing a specific tool on a specific server. Select tools only from the results obtained from the previous route each time.

When to use this tool:

- When using the route tool to route to a specific MCP server and tool

- When the 'execute-tool' fails to execute (up to 3 repetitions).

- When the user's needs and previous needs require the same tool.

Parameters explained:

-server_name: string, required. The name of the server where the target tool is located.

-tool_name: string, required. The name of the target tool to be executed.

-params: dictionary or None, optional. A dictionary containing all parameters that need to be passed to the target tool. This can be omitted if the target tool does not require parameters.

Prompt for evaluation

You are an expert in evaluating the performance of a tool-use agent. The agent is designed to help a human user use multi-tools to complete a task. Given the user's task, the agent's final response, key points for task completion, and tool call history, your goal is to determine whether the agent has completed the task and achieved all requirements.

Your response must strictly follow the following evaluation criteria!

***Important Evaluation Criteria*:**

1. You must carefully check whether the information (e.g. the coordinates of the addresses) comes from the tool call, if the agent get it from the internal knowledge, it should be considered failed.

2: Some tasks require to create files to be considered successful.

IMPORTANT

Format your response into two lines as shown below:

Thoughts: <your thoughts and reasoning process based on double-checking each key points and the evaluation criteria>

Status: "success" or "failure"

User Task: task

Key Points: key_points

Final Response: response

Tool Call History: tool_calls

Tool Descriptions: tool_descriptions

Prompt for identify key points

You are an expert tasked with analyzing a given task to identify the key points explicitly stated in the task description.

****Objective****: Carefully analyze the task description and extract the critical elements explicitly mentioned in the task for achieving its goal.

****Instructions****:

1. Read the task description carefully.
 2. Identify and extract ****key points**** directly stated in the task description.
- A ****key point**** is a critical element, condition, or step explicitly mentioned in the task description.
 - Do not infer or add any unstated elements.

****Respond with****:

- ****Key Points****: A numbered list of the explicit key points for completing this task, one per line, without explanations or additional details.""

task